DBMS COURSE PROJECT

Lost & Found Campus System

Relational Database Design using SQL Server (T-SQL)
Featuring Joins, Triggers, and Stored Procedures

SUBMISSION DATE	25 July 2026
PRESENTATION DATE	26 July 2026
DATABASE ENGINE	Microsoft SQL Server (SSMS)
PROJECT TYPE	Group Project (max. 4 students)

GROUP MEMBERS

Nabia Sajid	*Group Leader*
Muhammad Abdullah	*Member*
Maryam Muazzam	*Member*
Sathya Narayan	*Member*

1. Introduction

Students frequently lose personal belongings such as ID cards, wallets, phones, and water bottles on campus, with no centralized way to report or recover them. The **Lost & Found Campus System** solves this by giving students a shared database to report lost items, report found items, automatically surface likely matches between the two, request a claim on a found item, and have that claim verified by an administrator before the item is released.

The system is implemented as a single relational database in Microsoft SQL Server. It demonstrates three core DBMS concepts required for this project:

- **Joins** — combining data across related tables (e.g. showing an item together with the name of the student who reported it).
- **Triggers** — automatic actions fired by the database itself in response to data changes (e.g. updating an item's status the moment a claim is approved).
- **Stored Procedures** — reusable, named blocks of SQL logic that applications call instead of writing raw queries (e.g. a single call to review a claim).

2. Core Features

Feature	Description
Report Lost Item	A student records an item they lost — title, category, description, location, and date.
Report Found Item	A student (or staff) records an item they found, using the same structure.
Match Similar Items	The database automatically compares open Lost and Found items by category, location and date, and suggests likely pairs.
Claim Requests	A student requests ownership of a found item by submitting proof details.
Admin Approval	An administrator reviews each claim and approves or rejects it; approval finalizes the hand-over.

3. Entity-Relationship Diagram

The diagram below shows all five tables in the system, their columns, primary keys (PK), foreign keys (FK), and how they relate to one another.

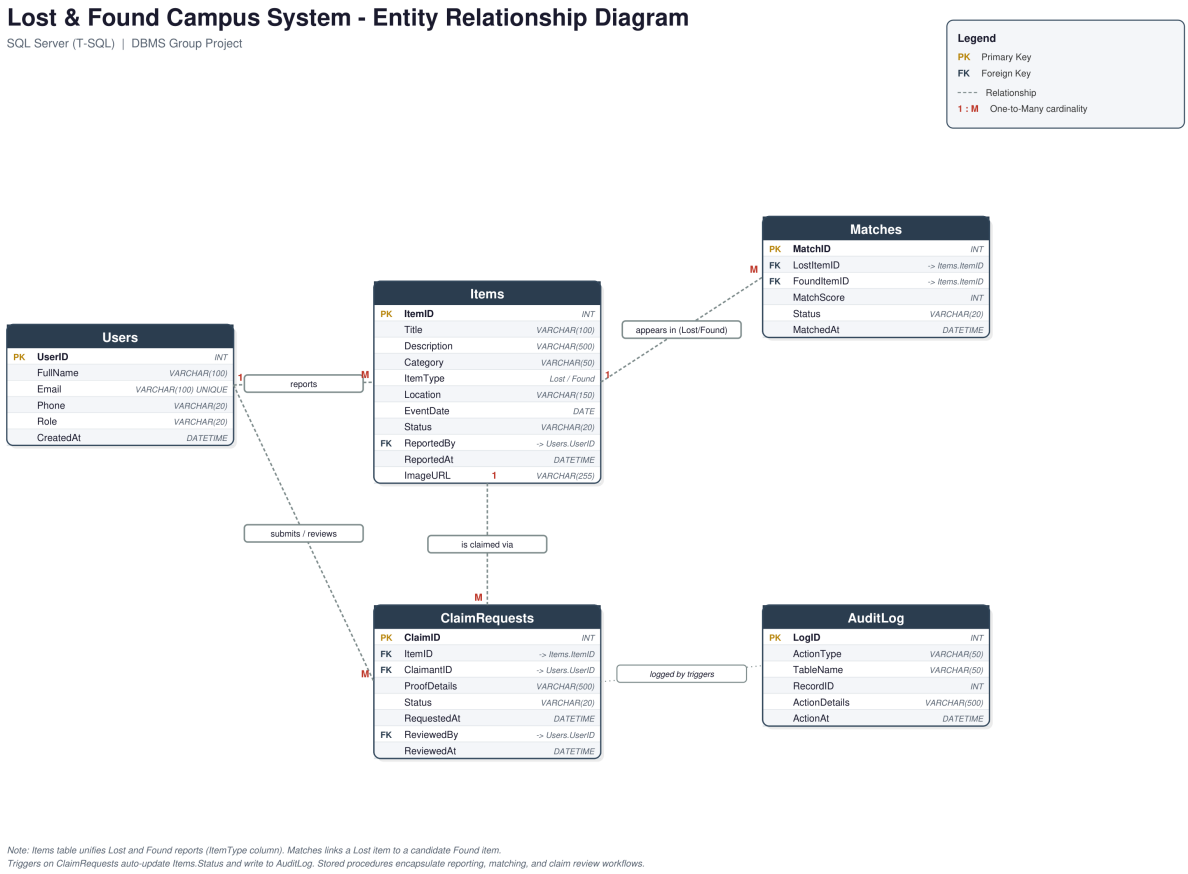

Figure 1 — Entity-Relationship Diagram of the Lost & Found Campus System

3.1 Table Overview

Table	Primary Key	Purpose
Users	UserID	Every student and admin account. Root of the ownership chain — items, claims, and reviews all trace back to a user.
Items	ItemID	A single unified table for both Lost and Found reports, distinguished by the ItemType column.
Matches	MatchID	Candidate pairings between a Lost item and a Found item, generated automatically by a stored procedure.
ClaimRequests	ClaimID	A student's request to claim a specific Found item, tracked through Pending → Approved/Rejected.
AuditLog	LogID	A trail of every automatic action the triggers perform, for accountability and debugging.

3.2 Why One Items Table Instead of Two?

Lost items and Found items share the exact same attributes (title, category, description, location, date). Rather than duplicate that structure into two separate tables, both are stored in one **Items** table with an **ItemType** column set to either 'Lost' or 'Found'. This keeps the schema smaller, avoids duplicated constraints, and lets the Matches table simply reference two rows in the same table — one Lost, one Found.

4. Database Schema

All tables are created with primary keys, foreign key constraints enforcing referential integrity, and CHECK constraints restricting status/type columns to valid values. A short excerpt is shown below; the complete DDL is in `01_schema.sql`.

```
CREATE TABLE Items (
    ItemID       INT IDENTITY(1,1) PRIMARY KEY,
    Title        VARCHAR(100)  NOT NULL,
    Category     VARCHAR(50)   NOT NULL,
    ItemType     VARCHAR(10)   NOT NULL CHECK (ItemType IN ('Lost','Found')),
    Location     VARCHAR(150),
    EventDate    DATE          NOT NULL,
    Status       VARCHAR(20)   NOT NULL DEFAULT 'Open'
                     CHECK (Status IN ('Open','Matched','ClaimPending','Claimed','Closed')),
    ReportedBy   INT NOT NULL REFERENCES Users(UserID)
);
```

4.1 Relationships Summary

Relationship	Cardinality	Meaning
Users → Items	1 : M	One user can report many items.
Items → Matches	1 : M	One item can appear in multiple suggested matches (as Lost or Found side).
Items → ClaimRequests	1 : M	One found item can receive multiple claim attempts.
Users → ClaimRequests	1 : M	One user can submit many claims, and separately review many claims as admin.

5. Joins

A **JOIN** combines rows from two or more tables based on a related column, so a query can return meaningful, human-readable results instead of raw foreign key numbers. The system uses joins throughout for reporting and dashboards. Two representative examples:

5.1 Lost Items with Reporter Details

Shows every lost item together with the name and email of the student who reported it, by joining **Items** to **Users** on the ReportedBy foreign key.

```
SELECT i.ItemID, i.Title, i.Category, i.Location, i.EventDate,
       u.FullName AS ReportedBy, u.Email
FROM Items i
JOIN Users u ON i.ReportedBy = u.UserID
WHERE i.ItemType = 'Lost';
```

ItemID	Title	Category	Location	Date	Reported By	Email
1	Black Wallet	Wallet	Library	2026-07-01	Ali Raza	ali.raza@campus.edu
2	iPhone 13	Electronics	Cafeteria	2026-07-03	Sara Khan	sara.khan@campus.edu
3	Blue Water Bottle	Bottle	Sports Ground	2026-07-05	Bilal Ahmed	bilal.ahmed@campus.edu

Output captured from SQL Server Management Studio.

5.2 Claim Requests with Item and Claimant Details

Joins three tables at once — **ClaimRequests**, **Items**, and **Users** — so an admin sees the item, the claimant, and the claim status in a single row.

```
SELECT c.ClaimID, i.Title AS ItemTitle, i.Category,
       u.FullName AS Claimant, c.Status, c.RequestedAt
FROM ClaimRequests c
JOIN Items i ON c.ItemID = i.ItemID
JOIN Users u ON c.ClaimantID = u.UserID;
```

ClaimID	Item Title	Category	Claimant	Status	Requested At
1	Wallet Found	Wallet	Sara Khan	Approved	2026-07-15 14:37

6. Triggers

A **trigger** is a block of SQL that SQL Server runs automatically whenever a specified event (INSERT, UPDATE, DELETE) happens on a table — the application never calls it directly. This project uses triggers to keep item status consistent without relying on the application layer to remember every side effect, and to build an audit trail for accountability.

Trigger	Fires On	What it does
trg_ClaimRequest_Insert	AFTER INSERT on ClaimRequests	Marks the item's Status as 'ClaimPending' the instant a claim is submitted, and writes an AuditLog entry.
trg_ClaimRequest_Approved	AFTER UPDATE on ClaimRequests	When a claim's Status changes to 'Approved', marks the item as 'Claimed' and automatically rejects any other pending claims on the same item.
trg_ClaimRequest_PreventSelfClaim	AFTER INSERT on ClaimRequests	Business rule: blocks a student from claiming an item they reported themselves, by rolling back the insert.

6.1 Example — Claim Approval Cascade

```
CREATE TRIGGER trg_ClaimRequest_Approved
ON ClaimRequests
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Status)
    BEGIN
        UPDATE i SET i.Status = 'Claimed'
        FROM Items i JOIN inserted ins ON i.ItemID = ins.ItemID
        WHERE ins.Status = 'Approved';

        UPDATE cr SET cr.Status = 'Rejected'
        FROM ClaimRequests cr JOIN inserted ins ON cr.ItemID = ins.ItemID
        WHERE ins.Status = 'Approved' AND cr.ClaimID <> ins.ClaimID
              AND cr.Status = 'Pending';
    END
END;
```

6.2 Verified Trigger Output

The sequence below was captured directly from SSMS to prove the triggers fire correctly during testing.

Step	Action	Column Affected	Result
1	Claim submitted for Item 4 (Wallet Found)	Items.Status	Open → ClaimPending
2	Admin approves ClaimID 1 via sp_ReviewClaim	ClaimRequests.Status	Pending → Approved
3	trg_ClaimRequest_Approved fires automatically	Items.Status	ClaimPending → Claimed

Resulting AuditLog entries (written automatically by the triggers):

LogID	Action	Table	Record ID	Details
1	INSERT	ClaimRequests	1	Claim request created for ItemID 4
2	UPDATE	ClaimRequests	1	Claim status changed to Approved

7. Stored Procedures

A **stored procedure** is a saved, named sequence of SQL statements that can be executed with a single call and accepts parameters, similar to a function in a programming language. Procedures keep business logic inside the database, so every application (web, mobile, admin panel) that uses this database gets the same guaranteed behaviour instead of re-implementing logic in each client.

Procedure	Parameters	Purpose
sp_ReportItem	@Title, @Category, @ItemType, @Location, @EventDate, @ReportedBy	Inserts a new lost or found report.
sp_MatchItems	(none)	Scans all open Lost/Found items and auto-generates candidate matches by category, location and a 7-day date window.
sp_ReviewClaim	@ClaimID, @AdminID, @Decision	Admin approves/rejects a claim inside a transaction; the triggers then cascade the status updates.
sp_GetUserReports	@UserID	Returns every item a given student has reported, most recent first.

7.1 Example — Auto-Matching Lost and Found Items

```
CREATE PROCEDURE sp_MatchItems AS
BEGIN
    INSERT INTO Matches (LostItemID, FoundItemID, MatchScore, Status)
    SELECT l.ItemID, f.ItemID,
           CASE WHEN l.Location = f.Location THEN 80 ELSE 50 END,
           'Suggested'
    FROM Items l
    JOIN Items f ON l.Category = f.Category
        AND l.ItemType = 'Lost' AND f.ItemType = 'Found'
        AND f.EventDate BETWEEN l.EventDate AND DATEADD(DAY, 7, l.EventDate)
    WHERE l.Status = 'Open' AND f.Status = 'Open'
      AND NOT EXISTS (SELECT 1 FROM Matches m
                      WHERE m.LostItemID = l.ItemID AND m.FoundItemID = f.ItemID);
END;
```

7.2 Verified Execution — EXEC sp_MatchItems

MatchID	Lost Item	Found Item	Score	Status
1	1	4	80	Suggested
2	2	5	80	Suggested
3	3	6	80	Suggested

All three lost items were correctly paired with their matching found items (matching wallet, phone, and bottle), each scoring 80 because the location matched exactly.

8. Testing & Verification Summary

The full workflow was executed end-to-end in SQL Server Management Studio to confirm every component works together correctly:

Test	Result	Notes
Schema creation	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "**PASS**" 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8.3, greek=0, italic=0, link=[], rise=0, text='PASS', textColor=Color(.117647,.517647,.286275,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	All 5 tables created with correct constraints and foreign keys.

Test	Result	Notes
Sample data load	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "**PASS**" 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8.3, greek=0, italic=0, link=[], rise=0, text='PASS', textColor=Color(.117647,.517647,.286275,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	5 users, 3 lost items, 3 found items inserted.

Test	Result	Notes
Join queries	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "**PASS**" 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8.3, greek=0, italic=0, link=[], rise=0, text='PASS', textColor=Color(.117647,.517647,.286275,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	All 6 join queries executed and returned correct combined data.

Test	Result	Notes
sp_MatchItems	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "**PASS**" 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8.3, greek=0, italic=0, link=[], rise=0, text='PASS', textColor=Color(.117647,.517647,.286275,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Correctly matched all 3 lost/found pairs by category and location.

Test	Result	Notes
trg_ClaimRequest_Insert	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "**PASS**" 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8.3, greek=0, italic=0, link=[], rise=0, text='PASS', textColor=Color(.117647,.517647,.286275,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Item status auto-changed to ClaimPending on claim submission.

Test	Result	Notes
sp_ReviewClaim + trg_ClaimRequest_Approved	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "**PASS**" 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8.3, greek=0, italic=0, link=[], rise=0, text='PASS', textColor=Color(.117647,.517647,.286275,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Claim approved, item auto-marked Claimed.

Test	Result	Notes
AuditLog	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "**PASS**" 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=8.3, greek=0, italic=0, link=[], rise=0, text='PASS', textColor=Color(.117647,.517647,.286275,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Both the insert and approval were logged automatically.

9. Conclusion

This project implements a fully working relational database for a campus Lost & Found system, demonstrating referential integrity through foreign keys, multi-table reporting through joins, automatic and consistent state management through triggers, and reusable business logic through stored procedures. All components were tested directly in SQL Server Management Studio and verified to behave as designed.